



Programmation Python

Filière LSI1

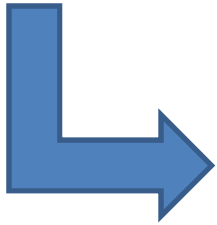


Chapitre I:

Environnement de développement Python 3

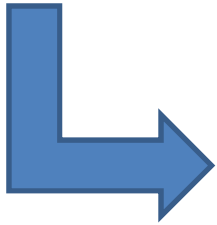
2020-2021 - SEMESTRE 2

nassima.medimegh@fsm.rnu.tn



Apprendre à programmer à l'aide
du langage Python

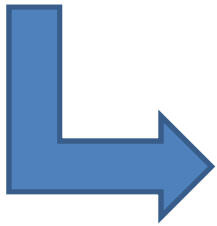
python Introduction



C'est quoi ?

- Langage de programmation de haut niveau.
- Créer en 1989 par Guido van Rossum.
- La première version publique de ce langage a été publiée en 1991.
- La dernière version de Python est la version 3. Plus précisément, la version 3.9 a été publiée en 7 décembre 2020 .
- Ce cours est basé sur Python 3.

python Introduction



Raisons du choix du langage ?

- Open source
- Multi-plateformes (exécuté sur différents systèmes d'exploitations)
- Syntaxe simplifiée
- Dispose d'un typage dynamique (créer des variables sans préciser le type, gestion automatique de la mémoire)
- Multi-paradigmes : notamment orienté objet, impératif, fonctionnel
- Distribué avec une riche bibliothèque standard



Installation de Python



- Afin de pouvoir développer en langage Python, vous devez installer les outils nécessaires :
 1. Télécharger et installer le langage Python depuis le site officiel Python.
 2. Télécharger et installer un **IDE Python** : de nombreux choix s'offre à vous : Pycharm, PyScripter, Wing.



Installation de Python

L'application IDLE de
Python
(*Integrated
Development and
Learning Environment*).

```
Python 3.8.0 Shell
File Edit Shell Debug Options Window Help
Python 3.8.0 (tags/v3.8.0:fa919fd, Oct 14 2019, 19:21:23) [MSC v.1916 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> # Mode interactif
>>> 2*125
250
>>> 2+
SyntaxError: invalid syntax
>>> 2+_
252
>>> s="bonjour"
>>> print(s)
bonjour
>>> """ message sur
plusieurs lignes

"""
' message sur\nplusieurs lignes\n\n'
>>> 1/0
Traceback (most recent call last):
  File "<pyshell#10>", line 1, in <module>
    1/0
ZeroDivisionError: division by zero
>>>
```



Les variables en Python



1- Qu'est-ce qu'une variable ?

- Une variable est définie par un **nom** dans un langage de programmation, alors que pour l'ordinateur il s'agit d'une **référence** désignant **une adresse mémoire** (i.e. un emplacement précis dans la mémoire).



Les variables en Python

2- Instructions de base sur les variables

- On attribue une valeur à une variable en utilisant l'opérateur d'affectation (=).
- Les variables sont utilisées sans être déclarées et leurs types dépendent de leurs contenus : **Typage dynamique**
- Une variable peut changer de type sans qu'il ait d'erreurs.
- Python est sensible à la casse, ce qui signifie que les variables `TesT`, `test` ou `TEST` sont différentes



Les variables en Python

2- Instructions de base sur les variables

```
>>> a=1
>>> X=1 # Affecter la valeur 1 à X
>>> Msg=" Python" # Affecter le message Python à la variable Msg
>>> a # Afficher la valeur de a
1
>>> print(Msg) # afficher le contenu du variable Msg
Python
>>> Msg
' Python'
>>> # Msg est de type string
>>> Msg = 3 # Msg change de type
>>> A=100
>>> # A et a sont differentes
>>> a
1
>>> A
100
>>> x= y =7 # affectation multiple
>>> x
7
>>> y
7
>>> a, b = 4, 7.35 # affectation parallèle
>>> a
4
>>> b
7.35
>>> |
```



Les variables en Python

3- Les types des variables

- Le **type** d'une variable correspond à la nature de celle-ci. Les types élémentaires dont nous aurons besoin dans un premier temps sont:
 - Les **entiers** (integer ou int),
 - Les **réels** (float)
 - Les **booléens** (bool)
 - Les nombres **complexes** (complex)
 - Les **chaînes de caractères** (str).



Les variables en Python

3- Les types des variables

A- Types élémentaires : classe int

- Le type int (entier) n'est pas limité en taille que par la mémoire de la machine.

```
>>> 20*5
100
>>> 2**4 # puissance
16
>>> 10/3
3.3333333333333335
>>> 10//3
3
>>> abs (5-10)
5
>>> 10%3 # Modulo
1
>>> |
```



Les variables en Python

3- Les types des variables

B- Types élémentaires : classe float

- un float est noté avec:
 - un point décimal (jamais avec une virgule)
 - un “e” symbolisant le “**10 puissance**”
- Les flottants supportent les mêmes opérations que les entiers
- Ils ont une précision infinie limitée

```
>>> 1.0 ;2.5 ;10.24
1.0
2.5
10.24
>>> 2.0e1
20.0
>>> 1.0e-1
0.1
```



Les variables en Python

3- Les types des variables

C- Types élémentaires : classe booléenne et complex

- **Type bool:**
 - Deux valeurs possibles: True, False
- **Type Complex:**
 - Les complex sont écrits en notation cartésienne formée de deux flottants.
 - La partie imaginaire est suffixée par j

```
>>> Msg ="Python"
>>> A==50 # test si A est égale à 50
False
>>> Msg == 'python'
False
>>> Msg
'Python'
>>> Msg== 'Python'
True
>>> A <= 100
True
>>> C= 3.5 + 2j
>>> C.real
3.5
>>> C.imag
2.0
>>> C
(3.5+2j)
... |
```



Les variables en Python

3- Les types des variables

D- Types élémentaires : [classe str](#)

- une chaîne de caractère est une séquence non modifiable, délimitée par " ou par '
- Les éléments d'une chaîne `s` sont indexés de 0 à **`len(s)-1`**, accessibles par l'opérateur `[]`
- On peut adresser les éléments d'une chaîne à partir de la fin avec des indices négatifs de **-1** à **-len(s)**
- On peut sélectionner des tranches d'une chaîne **`s[début : fin]`**
- Les chaînes peuvent être concaténées avec l'opérateur `+` et répétées avec `*`



Les variables en Python

3- Les types des variables

```
>>> S= "Chaine de caractère"
>>> type (S)
<class 'str'>
>>> S[0]
'C'
>>> S[1]= 'c' #Une fois une chaîne de caractères définie, vous ne pouvez plus m
odifier un de ses éléments.
Traceback (most recent call last):
  File "<pyshell#57>", line 1, in <module>
    S[1]= 'c' #Une fois une chaîne de caractères définie, vous ne pouvez plus m
odifier un de ses éléments.
TypeError: 'str' object does not support item assignment
>>> S[len(S)-2]
'r'
>>> S[1:6]
'haine'
>>> S +" Python"
'Chaine de caractère Python'
>>> S + "Python" #Concaténer
'Chaine de caractèrePython'
>>> S *2 # répéter 2 fois S
'Chaine de caractèreChaine de caractère'
```



Les variables en Python

3- Les types des variables

Exercice :

Testez les lignes d'instructions suivantes

```
>>> r , pi = 12, 3.14159
```

```
>>> s = pi * r**2
```

```
>>> Ch= "bonjour"
```

```
>>> print (s)
```

```
>>> Ch=s
```

```
>>> type(r), type(pi), type(s), type(Ch)
```

```
>>> r , pi = 12, 3.14159
>>> r
12
>>> s = pi * r**2
>>> s
452.38896
```

```
>>> print (s)
452.38896
>>> type (r)
<class 'int'>
>>> type (s)
<class 'float'>
```

```
>>> type(pi)
<class 'float'>
>>> Ch ="bonjour"
>>> Ch=s
>>> type (Ch)
<class 'float'>
```


python Opérations élémentaires

- Les opérateurs sont utilisés en Python pour effectuer des opérations sur les variables et les valeurs associées.
- Python classe les opérateurs selon les groupes suivants :
 - Entrées/Sorties
 - Opérateurs arithmétiques
 - Opérateurs d'assignation
 - Opérateurs de comparaison
 - Opérateurs logiques



Opérations élémentaires

1- Entrées/Sorties

- Le langage Python possède déjà des fonctions prédéfinies :
 - La fonction **print** Python est souvent utilisée pour générer la sortie des variables.
 - La fonction **input** permet de récupérer un str à partir de l'entrée standard. (lire une saisie par clavier)
 - Il est possible de convertir l'**str** résultant en une valeur numérique en utilisant les constructeurs int, float ou complex, par exemple pour lire un nombre flottant :



Opérations élémentaires

1- Entrées/Sorties

```
>>> msg= input()  
Bonjour  
>>> print(msg, type(msg))  
Bonjour <class 'str'>  
>>> input("donner un entier ")  
donner un entier 8  
'8'  
>>> a=input("donner un entier ")  
donner un entier 2  
>>> float(a) # conversion de type  
2.0
```

```
>>> X= float(input(" donner x : "))  
donner x : 5  
>>> X  
5.0  
>>> type(X)  
<class 'float'>  
>>> |
```



Opérations élémentaires

2- Les opérateurs arithmétiques

- Les opérateurs arithmétiques sont utilisés en Python pour effectuer des opérations de calcul sur les variables comme addition, multiplication, division...

Opérateur	Description
'+'	addition
'-'	soustraction
'*'	muliplication
'/'	division
'%'	modulo (reste de la division euclidienne)
'**'	Exponentiation
'//'	quotient de la division euclidienne

priorité	opérateur	signification	exemples
1	()	parenthèses	$2^{**}1/2$ donne 1.0 par contre $2^{**}(1/2)$ donne $\approx \sqrt{2}$
2	**	puissance	$1j^{**}2$ donne $(-1+0j)$
3	+ et -	signe	-- 3 done 3
4	*	multiplication	$2*4$ donne 8 et $2.0 * 4$ donne 8.0
	/	division	$3/2$ donne 1.5, $2/(2+0j)$ donne $1+0j$
	//	division euclidienne	$9//2$ donne 4 $2.75//0.5$ donne 5.0
	%	reste de la division euclidienne	$9\%2$ donne 1 $2.75\%0.5$ donne 0.25
5	+	addition	$3 + 4j + 5.5$ donne $8.5+4j$
	-	soustraction	$3-10$ donne -7



Opérations élémentaires

3- Les opérateurs d'assignation

- Les opérateurs d'assignation sont utilisés en Python pour assigner des valeurs aux variables :

Opérateur	Exemple	Explication
=	x = 7	x prends la valeur 7
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
*=	x *= 5	x = x * 5
/=	x /= 5	x = x / 5
%=	x %= 5	x = x % 5 (reste de la division euclidienne de x par 5)
//=	x //= 5	x = x // 5 (quotient de la division euclidienne de x par 5)
**=	x **= 3	x = x ** 3 (x^3 ie x*x*x)



Opérations élémentaires

3- Les opérateurs d'assignation

- Il suffit qu'un seul float apparait dans une expression arithmétique d'entiers pour que le résultat final devienne float. On dit que les floats sont contagieux par rapport aux entiers.
- Il suffit qu'un seul complex apparaisse dans une expression arithmétique pour avoir un résultat final de la classe complex. Les complex sont contagieux par rapport aux autres classes numériques.



Opérations élémentaires

4- Les opérateurs de comparaison

- Les opérateurs de comparaison sont utilisés en Python pour comparer les variables
- Le résultat est généralement une valeur de vérité True ou False

Opérateur	Description
<code>==</code>	opérateur d'égalité
<code>!=</code>	opérateur différent
<code>></code>	opérateur supérieur
<code><</code>	opérateur inférieur
<code>>=</code>	opérateur supérieur ou égale
<code><=</code>	opérateur inférieur ou égale



Opérations élémentaires

5- Les opérateurs logiques

- Les opérateurs logiques sont moins prioritaires que les opérateurs relationnels.

Opérateur	Description
and	et logique
or	ou logique
not	Négation logique

exemples
not true donne False
x and y donne True uniquement si x==y=True
x or y donne False uniquement si x==y=False

Exercice

- Assignez les valeurs respectives 3, 5, 2 à trois variables a, b, c
- Effectuez l'opération $a - b/c$

python Structures de contrôles

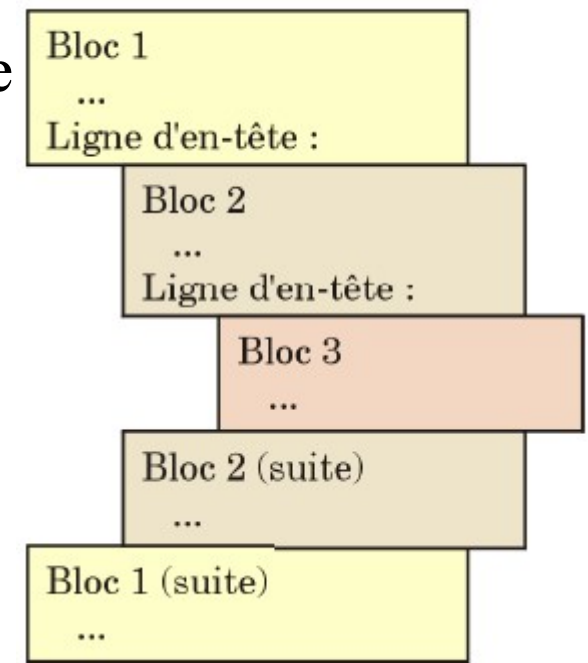
- Notions fondamentales.
- Structures conditionnelles.
- Structures itératives.



Structures de contrôles

1- Notions fondamentales

- **Un bloc** est un groupe d'instructions formant une seule unité comportementale. Un bloc est formé par au moins une instruction et peut contenir d'autres sous-blocs $\Rightarrow$ blocs imbriqués.
- **Indentation** consiste à décaler les instructions d'un bloc imbriqué vers la droite en utilisant des caractères blancs (espaces et/ou tabulations). Les instructions alignées sur la même verticale appartiennent au même bloc.





Structures de contrôles

2- Structures conditionnelles

- Permet de choisir un bloc à exécuter conditionnellement. Le flot d'exécution est branché au bloc dont la condition associée est vraie.

A- La structure sélective **if ... else ...**

- La structure sélective **if ...else**, permet d'exécuter un ensemble d'instructions lorsqu'une condition est réalisée.

- Syntaxe :

```
1 if(condition):  
2     instructions ...  
3 else:  
4     autres instructions...
```

```
2 age = 19  
3 if(age >= 18):  
4     print("Vous êtes majeur !")  
5 else:  
6     print("Vous êtes mineur !")  
7 # affiche vous êtes majeur
```



Structures de contrôles

2- Structures conditionnelles

B- L'instruction elif

- L'instruction **elif** est employée généralement lorsque l'exception comporte 2 ou plusieurs cas à distinguer.

```
>>> maximum = x if x>y else y
>>> if x>y :
...     maximum = x
... else :
...     maximum=y
```

```
age = int(input('tapez votre age : '))
if (age >= 18):
    print("Vous êtes majeur !")
elif (age<15):
    print("Vous êtes trop petit !")
else:
    print("Vous êtes adolescent!")
```



Structures de contrôles

3- Structures itératives

A- La structure répétitive For ...

- La boucle for, permet d'exécuter des instructions répétées à un certain nombre de fois. Sa syntaxe est :

```
for compteur in range(début_compteur, fin_compteur):  
    instructions|...
```

```
>>> for i in range(1,11) :  
        print (i)  
  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
>>> for i in range (3) :  
        print (i)  
  
0  
1  
2  
>>> |
```



Structures de contrôles

3- Structures itératives

B- La structure répétitive while

- Permet d'exécuter un ensemble d'instructions tant qu'une condition est réalisée et que l'exécution s'arrête lorsque la condition n'est plus satisfaite.
- Utilisée généralement lorsque le nombre de répétition n'est pas connu à l'avance.
- Sa syntaxe est : `while ( condition ):`

`instructions...`

```
i = 1
while ( i <= 10 ) :
    print(i)
    i = i + 1
```




Structures de contrôles

Exercice

```
>>> a, b, c = 1, 1, 1
>>> while c < 11 :
...     print b,
...     a, b, c = b, a+b, c+1
```

```
1 2 3 5 8 13 21 34 55 89
```



Structures de contrôles

3- Structures itératives

C- Les instructions "break", "continue" :

- L'instruction "break" permet d'arrêter une boucle avant sa fin.
- L'instruction "continue" est similaire, mais au lieu d'interrompre la boucle, elle permet de passer à l'itération suivante.

```
for i in range(5):  
    if i==3:  
        break  
    print i
```

affichera 0 | 2

tandis que

```
for i in range(5):  
    if i==3:  
        continue  
    print i
```

affichera 0 | 2 4

- **Exercice 1.** Ecrire un programme en Python qui demande à l'utilisateur de saisir deux nombres a et b et de lui afficher leur maximum
- **Exercice 2.** Ecrivez un programme qui affiche les 20 premiers termes de la table de multiplication par 7.

```
1 * 7 = 7
2 * 7 = 14
3 * 7 = 21
4 * 7 = 28
5 * 7 = 35
6 * 7 = 42
7 * 7 = 49
8 * 7 = 56
9 * 7 = 63
10 * 7 = 70
11 * 7 = 77
12 * 7 = 84
13 * 7 = 91
14 * 7 = 98
15 * 7 = 105
16 * 7 = 112
17 * 7 = 119
18 * 7 = 126
19 * 7 = 133
20 * 7 = 140
```

■ Exercice 1.

```
>>> c=0
>>> while c < 20 :
    c=c+1
    print(c, " * 7 = ", c*7)
```

■ Exercice 2.

```
>>> a= input (" donner un nombre  a = " )
    donner un nombre  a = 10
>>> b= input (" donner un nombre  b = " )
    donner un nombre  b = 120
>>> print ("le maximum est a= ",a) if a>b else print ("le maximum est b= ",b)
le maximum est b= 120
... |
```